

it felt like the feature was tacked on. Ruby was his attempt at such a language. The result of that decision is that everything is an object in Ruby. While some languages differentiate between ‘raw’ data types such as integers and floats, and more complex object-oriented datatypes it supports, that is not the case with Ruby. Even integers and floats are objects which allows for odd looking things such as calling methods on integer or float values (`3.1412.to_s()` for instance will convert the number 3.1412 to a string).

Another aspect of everything being an object in Ruby, is that even your own code is an object that can be manipulated. In other words, Ruby supports meta-programming. Even classes and functions are objects in Ruby, and can as such be created, modified, passed around and used as return values in Ruby.

Ruby even has ‘open’ classes, so a class created elsewhere, by someone else, in a different module can be edited by you in your own code to add features. This is not the same as subclassing, which is also supported, you are modifying the class itself. You can even modify core classes, such as the inbuilt Integer class to add features to integer numbers. For instance you could add an ‘`is_prime`’ method to the Integer class, and then be able to quickly check if any number is prime as `17.is_prime` or `9.is_prime`.

Given what we’ve just said about how easy Ruby makes it to modify things, it might seem odd that Ruby also supports functional programming. Ruby goes through the trouble of marking any method that has side-effects. Any method on an object that changes the internal state of that object is marked with a ‘!’ at the end. So if you have an array called ‘`arr`’, calling ‘`arr.sort`’ will return a sorted version of the array, while ‘`arr.sort!`’ will sort the array itself in place. This allows people who wish to program in a functional paradigm to do so.

Where Python takes the view that it is best to provide one way to do things (i.e. not have many alternate forms of syntax for the same thing) Ruby takes the opposite view, that having different ways of doing the same thing is good. Sometimes doing things a different way makes them clearer if not better. Which view is better? We say that’s a silly question, and its silly to rank subjective views on an objective scale.

Significance

If we had one word to explain the significance of Ruby, that word would be ‘Rails’. Ever since the release of ‘Ruby on Rails’ in 2005, the framework has taken the world of web development by storm. Like it or hate it, it’s impossible to deny that it has had a huge impact. In fact it was initially the framework of choice of Twitter — they have since moved to other technologies — and today